DEVBRIDGE M5 · DESIGN RECORD

CV Verification Engine

Verifies developer CVs against the systems the candidate names — GitHub, Codeforces, LeetCode, CodeChef, live deployments — extracting typed claims with verbatim spans and running 28 deterministic checks.

> It produces **no score** and **no verdict**. It reports evidence and says explicitly what it could not check. A person decides what any of it means.

29/29	547	10,639	17/17
checks built, every one in scope	tests passing	lines across 32 modules	mutation detection, 0 collateral

CONTENTS

01 Status
02 Architecture
03 The contract
04 Ingest
05 Span grounding
06 Extraction
07 The gate
08 Dates & intervals
09 Evidence
10 The rules
11 The report
12 Narrative
13 Worksheet
14 The handoff to M6
15 Interfaces
16 Cross-cutting
17 Measurement
18 Not built
19 Open decisions
20 The invariant
21 A worked example

01 Status

Twenty-nine of a catalogue of forty-one checks — every one in scope. Seven are human by design, one is excluded on fairness grounds, and the four credential checks are out of scope for this project.

GROUP	BUILT	NOT BUILT	REASON
D document-intrinsic	D2 D3 D4 D5 D6 D7 D8 D9 D10 D11 D12 — **11/11**	—	complete
G GitHub	G2 G3 G4 G5 G6 G7 G8 G9 G10 G11 — **10/10**	—	complete
P platforms	P1 P2 P3 P4 P5 P6 — **6/6**	—	complete; P5 is scraped, so attention-only
W deployments	W1 — **1/2**	W2	human by design — a reviewer opens the link
C credentials	C5 — **1/5**	C1–C4	out of scope for this project
H human	—	H1–H6	human by design

Measured behaviour, not asserted

`cvv evaluate` corrupts exactly one fact in a real document and asks whether the check responsible notices.

DOCUMENT	CLAIMS	COVERAGE	DETECTION	COLLATERAL
student CV — IIT Kharagpur, 21 Oct	19	57%	**17/17**	0
synthetic CV — VIT Vellore, 15 Sep	20	44%	**7/7**	1 (correct)
professional CV — 2 pages, 15 Sep	15	22%	**2/2**	0

> **Collateral is the number that matters** — rules firing on a mutation they had no business noticing. A system that flags six things when one number changes is noisy, not sensitive, and every extra finding lands on a real person. It started at 2; both were real bugs invisible to 294 passing tests.

Test distribution

FILE	TESTS	COVERS
test_rules.py	170	all 28 checks, against recorded evidence
test_github.py	44	authored-file detection, repo evidence, manifests
test_contract.py	44	our models and a whole emitted report vs. the JSON Schemas
test_dates.py	36	the date parser's table
test_api.py	28	the HTTP surface
test_store.py · test_narrative.py	22	both store backends · the second LLM site
test_intervals.py · test_filters.py	21	three-valued interval algebra · the deterministic gate
test_ingest.py	21	PDF · DOCX · OCR · PDF dates to ISO 8601
test_extract.py	19	extraction internals
test_logs.py	18	log redaction
test_spans.py	15	grounding and the numeral guard
test_codechef.py	14	the scraper, against a synthetic page
test_structure.py	13	architectural rules, enforced
test_leetcode · test_worksheet	12–11	connector · manual prep
test_gather · test_evaluate · test_cli · test_reference	9–3	account choice · the harness · commands · roll schemes

02 Architecture

One package, one process. The tree reads in execution order, and each arrow is a typed contract.

```
ingest  →  extract  →  gather  →  rules  →  views
 bytes     ClaimGraph  EvidenceBundle  FindingsReport  what a person reads
```

That is what makes every stage testable alone: `rules` never sees a PDF, `narrative` never sees a URL.

Two structural rules, enforced rather than asserted

`rules.py` imports only pure modules. No `httpx`, no `gather`. If a rule needs data it arrives in the `EvidenceBundle` or the rule abstains. A test walks the AST and holds the allow-list:

```
PURE = {
    "cvv.models",      # the contract
    "cvv.intervals",   # the date algebra
    "cvv.ids",         # stable identifiers, no I/O
    "cvv.filters",     # pure predicates over claims
    "cvv.reference",   # curated tables loaded from data/
    "cvv.logs",        # writes to stderr, gathers nothing
}
```

The test also fails on a *stale* entry, so the list cannot rot. Before it existed, the project's own documentation claimed this enforcement and it was not there — and the rule it described had already drifted.

`narrative.py` cannot reach the document. The faithfulness guarantee is the function signature, not the prompt: an endpoint that cannot receive the source text cannot leak it into prose, whatever the model is asked to do.

03 The contract

Nine claim kinds, from the authoritative JSON Schema. `StrEnum` throughout, so a typo is an `AttributeError` rather than a silently false comparison.

```
class ClaimKind(StrEnum):
    EMPLOYMENT · EDUCATION · CERTIFICATION · PROJECT · SKILL
    PROFILE              # carries the handle; 18 checks depend on it existing
    ACHIEVEMENT          # carries the directly checkable numeric
    IDENTIFIER           # the roll number → D9
    DECLARATION_MARKER   # the institution's own disclaimer
```

A CV date denotes a set, not a point

```
class FuzzyDate(Model):
    year: int = Field(ge=1950, le=2100)
    month: int | None = None
    day: int | None = None
    precision: Precision          # exact | month | year
```

Validators enforce that precision matches the components present, and that an exact date is a real calendar day — 31 April would otherwise raise inside `intervals.bounds()` during rule evaluation, far from the input that caused it.

The validator that catches most integration breakage

```
@model_validator(mode="after")
def _spans_index_their_verbatim(self, info) -> ClaimGraph:
    text = self.source.normalised_text
    for c in self.claims:
        p = c.provenance
        if p.char_end > len(text):
            raise ValueError(f"{c.claim_id}: span exceeds source text")
        if p.recovery is Recovery.EXACT and text[p.char_start:p.char_end] != p.verbatim
            raise ValueError(f"{c.claim_id}: span does not index its verbatim")
```

What the findings model deliberately lacks

There is no `overall_score` field and there never will be. A single number invites a threshold, a threshold becomes a filter, and a filter rejects people for reasons nobody can inspect.

Two validators carry design decisions rather than type checks:

```
# An attention finding is UNCONSTRUCTABLE without its benign reading.
if self.tier is Tier.ATTENTION and not self.innocent_explanation:
    raise ValueError("tier=attention requires innocent_explanation")

# Abstention without a reason is exactly the silence the design forbids.
if self.status in (ABSTAINED, ERROR) and not self.reason:
    raise ValueError(f"status={self.status} requires a reason")
```

04 Ingest

Three paths, dispatched on extension and on whether a PDF actually carries text.

INPUT	PATH	NOTES
`.pdf`	PyMuPDF, page by page	producer and creation metadata retained for D4
`.pdf`	< 120 chars/page → **OCR**	RapidOCR, opt-in extra; report carries a caveat banner
`.docx`	python-docx, walking body XML	tables flattened **in document order**
`.doc`	refused, with its reason	a different binary format; guessing loses content

Normalisation owns the definition of "the document"

NFKC folds what a PDF quietly inserts — ligatures (fi → fi), non-breaking spaces, full-width digits — any of which makes an exact span match fail on text a human would call identical. An index map carries offsets back to the raw string, because after folding they no longer line up and provenance must point at the original.

OBSERVED — DOCX

`Document.paragraphs` silently omits every table. A CV whose education sits in a two-column layout reads as headings with nothing under them. It does not fail: it grounds cleanly and loses half the claims.

RESOLVED

The body's XML children are walked directly, and each table **row becomes one line** — a row is one line of a CV, and splitting it would make `line_at` cite a cell rather than what a reader sees.

OBSERVED — OCR

The reference CV rasterised at 150 dpi: 3,910 characters at **0.981 mean confidence**. Inside that — `ASHUTOSHRANJAN`, `IITKharagpur`, `KrishnaPublicSchool`. Word boundaries fail, not characters.

RESOLVED

Grounding still holds — a verbatim is recovered from the same string. What degrades is reference-table matching, so those rules abstain rather than contradict. The report prints a banner above every bucket so a reader knows why.

05 Span grounding

The model returns a verbatim; this module verifies that text really is in the document and returns where. Without it, a model that reads "rating of 1495" and returns 1595 produces a decisive contradiction against a claim the CV never made.

Two tiers, and the second needs a guard

```
def numerals_present(verbatim: str, text: str) -> bool:
    """Every digit sequence in the verbatim must occur in the document, as a whole run.
    present = set(_NUMERALS.findall(text))
    return all(n in present for n in _NUMERALS.findall(verbatim))
```

> **Measured:** a hallucinated `rating of 1800` where the document says `1495` scores **0.944** on partial-ratio and sails through a bare 0.92 threshold — three characters out of fifty-three. Similarity is blind to precisely the character that matters, and numbers are where the decisive checks live.

Whole runs, not substrings. A substring test lets "rating of 149" through against a document saying 1495 — dropping a digit is a one-character edit, exactly what the fuzzy tier forgives and exactly what this guard exists to refuse.

The window is a boundary, not a hint

```
if window is None:
    lo, hi = 0, len(text)
else:
    lo = max(0, window[0] - WINDOW_MARGIN)   # 64 chars of slack
    hi = min(len(text), window[1] + WINDOW_MARGIN)
```

A margin, not a fallback. Given a 180-character education section, the model returned "SKILLS AND EXPERTISE" and "EXTRA CURRICULAR ACTIVITIES" as degrees; a document-wide search found that text elsewhere and grounded both.

Token-shaped fields are recovered, not trusted

Handles, repo owners, credential ids, roll numbers and URLs must each be a literal substring of the document, **taking the document's casing**. A model works in tokens and normalises: it returns `ar-009` for `AR009`, the API 404s, and the report says a real account does not exist.

The search is scoped to the claim's own span with no fallback. A real CV carried `handle: AR009` (Codeforces) and `handle: ar009` (CodeChef) two lines apart; a document-wide search returned whichever appeared first.

06 Extraction

Eight narrow passes, each over a scoped window. The model proposes; deterministic code decides what is real.

PASS	WINDOW	PRODUCES
`achievements`	routed section	platform · metric · value · qualifier · contest
`projects`	routed section	title · urls · dates · technologies
`employment`	routed section	org · title · type · dates
`education`	routed section	degree · org · dates
`skills`	routed section	name + category + years, per technology
`summary`	**top 15%** of the document	total years of professional experience
`declarations`	**bottom 20%**	the institution's own disclaimer
`handles`	routed section	platform + handle, **no verbatim**

`required` is the lever

A single call with every field optional returned 18 claims carrying `kind` and `verbatim` and nothing else — the model segmented the document and populated none of it. Four narrow calls with `required` in the JSON Schema returned every value correctly. Grammar-constrained generation makes an optional field one a small model will skip.

The model routes the sections, not a keyword list

OBSERVED

`sections=("ACHIEVEMENT", "AWARD")` encodes what *one* CV called its section. A second document headed it **"COMPETITIVE PROGRAMMING"**, matched nothing, fell back to the whole document, and lost two of three ratings and the only contest rank — silently, on a CV that had done nothing wrong.

RESOLVED

The model is shown the headings and says what each holds; the answer is verified against headings the document actually has. Keyword lists survive only as a fallback, so routing can improve a window and never break one.

Two refinements each cost a debugging round:

- **One window per *contiguous* run, never one overall.** The reference CV routes both `EDUCATION` and `COURSEWORK INFORMATION` to education — correctly — and they sit at opposite ends of the page. Spanning them turned a 180-character window into a 4,252-character one. Choosing between them was the next wrong answer: longest-first picked COURSEWORK and lost every degree.
- **A category the model needs must exist even if no pass consumes it.** With no `coursework` category, coursework went to `education`, and the education pass turned "Computer Science" into a degree awarded by "University".

Ask the right question

The skills pass returned the CV's own category headings as skills — `programming languages`, `utilities` — and the plan was an ontology lookup to filter them. Not needed: asking *"what category is each item"* instead of *"list the skills"* made both models exclude headings by themselves, because a heading has no plausible category. **Junk output is often a task-framing problem before it is a capability one.**

Do not ask for a verbatim you can derive

The `skills` and `handles` passes return no verbatim at all. Asked for both a name and a verbatim, the model sets them equal and returns the whole line as the skill name; asked for a handle and a verbatim it *composes* one — it returned `"Codeforces: AR009"` for a document that says `"(handle: AR009)"`, and grounding dropped it along with ten other proposals.

A handle is then located **on a line that names its own platform**. This CV carries `AR009` on its Codeforces line and `ar009` on its CodeChef line, so a window-wide search gave *both* claims the Codeforces line the moment the model miscased one. Every field individually real; the pairing silently wrong.

What still uses a regex, and why

Five patterns became model passes because each encoded how *one* document happened to be phrased:

PATTERN	RETURNED NOTHING ON
`round\s+#?(\d{2,4})`	AtCoder, LeetCode, ICPC, even "Round 2"
`\d{2}[A-Z]{2}\d{5}`	every institution but IIT Kharagpur
`could not verify|self-declared`	any institution wording its footer differently
`(platform)…handle:\s*(\w+)`	`Codeforces: ar009`, a bare URL, a bracketed name
`(\d+) years of experience`	"8 yrs", "since 2016", any other language

Two remain and both earn it. `GITHUB_REPO`, because a URL's grammar is declared by the internet and not by a CV. `PLATFORM_MENTION`, which is *enforcement* rather than parsing and is built **from** the contract's `Platform` enum so it cannot drift out of it.

Roll numbers are still found by pattern, but the patterns come from the same data file that interprets them — adding VIT Vellore as a data entry made `21BCE1847` decode end to end with no code change.

07 The deterministic gate

Grounding proves a claim's text is real. It does not prove the model classified that text correctly — and misclassification produces false decisive findings just as readily as hallucination.

OBSERVED — THE HEADLINE FAILURE

Given the whole CV, the model found one achievement — Codeforces 1495 Specialist — and emitted those same field values against **fourteen different verbatims**, including the CV's name header.

WHY GROUNDING CANNOT CATCH IT

Every verbatim is genuine and 1495 is a genuine rating. Only the *pairing* is invented. Checking the document is not enough — the number has to be in *this* span.

```
def value_is_in_its_own_span(claim: Claim) -> bool:
    if claim.value is None:
        return True
    return numbers_in(str(claim.value)) <= numbers_in(claim.provenance.verbatim)
```

As whole numbers, with grouping commas removed. Version 1 asked `str(value) in verbatim`, so a value of 10 "appeared" in "Ranked 1106" — a model truncating a number passed the very check that exists to catch it.

FIELD	RULE
`value`	the number must appear in *this* claim's verbatim
`platform`	whichever the verbatim names; none named → unattributable, dropped
`proficiency`	the band the sentence uses, or `None`. Never invented
two or more platforms named	flag `aggregate_across_sources`; the rule abstains

The model labelled a CodeChef rating `codeforces`, which would have compared 1733 against Codeforces' 1495 and fired a decisive contradiction against an honest candidate. It also labelled four skills `expert` — a word appearing nowhere in the CV.

The other three stages

- **Excluded exams** — read from the verbatim, never the model's label.
- **Semantic deduplication**, keyed on meaning rather than span: the model returns the same claim with slightly different verbatims, which ground to different offsets and survive a span-keyed dedup. Five copies of one rating got through that way.
- **One region, one kind.** Two *container* claims over the same text is a classification conflict, and the more specific kind wins.

The drop counts are not decoration — they are the extraction-quality metric, and they travel into the report.

08 Dates and intervals

No `dateutil`: it is permissive by design and reads `2022-26` as June 2022 — a confidently wrong precision, which is worse than a failure.

The slash-date problem

OBSERVED

A third CV had five roles in `MM/DD/YYYY`. All five parsed to nothing, and D5, D10, D11 and D12 all abstained "fewer than two dated roles" — **the entire employment group disabled by one date format.**

RESOLVED

Nothing in `02/05/2024` says which field is the month. The *rest of the document* does: a CV writing `01/26/2024` anywhere has shown its hand, because 26 is not a month.

Where nothing is decisive — or where the evidence conflicts — the date parses to the **year**. The year is certain, the month is a coin toss, and a wrong month can raise a DEFINITE overlap and contradict somebody where a year-precision interval never can.

Three-valued interval algebra

SPAN	DEFINITION	MEANING
core	`[start.latest, end.earliest]`	covered under *every* reading
max	`[start.earliest, end.latest]`	covered under *some* reading

```
cores intersect          → DEFINITE     (scorable)
only max spans intersect → POSSIBLE     (context, never scored)
neither                  → NONE
```

> **Only DEFINITE may raise a scoring discrepancy.** A single-year interval has an *inverted, empty* core — "2021" has `start.latest` 2021-12-31 after `end.earliest` 2021-01-01 — and can never be DEFINITE. That emptiness is the mechanism, not a defect.
>
> Live example: CF-Connect `[May '25 – Jun '25]` against Dijkstra Visualizer `[Jun '25]`. Both cover June, so a naive comparison reports a contradiction; the two could equally have run Jun 1–10 and Jun 20–30, and the document does not say.

`today` is injected everywhere. Nothing reads the clock — a verification must reproduce when re-run, and a report stored in March must still explain itself in September.

09 Evidence gathering

Every outbound request lives behind one module. Every failure path writes a **typed gap** and gathering continues: one dead source must never fail the bundle.

CONNECTOR	SOURCE	FEEDS
Codeforces	public API, no auth	`P1 P2 P3`
LeetCode	public GraphQL, no auth	`P4`
GitHub	REST + GraphQL contributions calendar	`G2–G11`
HTTP liveness	one GET per claimed deployment	`W1`
CodeChef	the public profile page, scraped — no API exists	`P5`

CodeChef's evidence is marked `source_quality: scrape`, and P5 is attention-only for that reason. A field the page lacks is `None`, an unrecognised page is a gap, and a missing handle is a finding only when CodeChef redirects the profile URL to its homepage — any other redirect means the site changed, and says so.

Server-side timestamps only

`git commit --date=` and `GIT_COMMITTER_DATE` are attacker-controlled. Only `repo.created_at`, `pushed_at` and the contributions calendar are trustworthy; where no server-side value exists the field stays `null` and a gap opens. `RepoEvidence.first_commit` is therefore permanently `None`.

Two connector details that are easy to get wrong

- **LeetCode's `acSubmissionNum` carries "All" alongside the per-difficulty rows.** A naive sum gives `253 + 60 + 141 + 52 = 506` — exactly double. That inflates the *observed* count, making an honest claim look understated.
- **LeetCode publishes a live rating and no peak**, so it is stored as `current_rating` with `max_rating` left `None`. Storing it as a peak would let P4 call someone an overstater for a rating that has since fallen.

10 The rules

Twenty-seven checks, each a frozen dataclass registered by decorator, each declaring its tier and whether it may move a number.

Three control signals

SIGNAL	MEANING
`yield Discrepancy(…)`	a finding
`raise Abstain(reason)`	could not run — the reason reaches the reader
`raise Corroborated(note)`	nothing to raise, something to **confirm**

`Corroborated` exists because some checks can only ever confirm. GitHub organisation membership is **private by default**, so its absence is the ordinary state of an honest employee and its presence is evidence from a party other than the candidate.

The catalogue

ID	VER	TIER	SCORES	WHAT IT CHECKS
D2	2.0	ATTENTION	✗	claims dropped, split by cause
D3	1.0	ATTENTION	✗	the document's own disclaimer
D4	1.0	ATTENTION	✗	file provenance — edits after generation
D5	2.1	DECISIVE	✓	two roles held at once
D6	1.0	ATTENTION	✓	stated total vs. the dated roles
D7	1.0	DECISIVE	✓	a project dated before its technology existed
D8	1.0	ATTENTION	✓	degree span vs. standard length
D9	2.0	DECISIVE	✓	roll-number structure vs. programme and dates
D10	2.0	DECISIVE	✓	one stint of a role dated two irreconcilable ways
D11	1.1	ATTENTION	✗	gap between roles
D12	1.0	ATTENTION	✗	seniority against tenure
G2	1.0	DECISIVE	✓	the account exists
G3	1.0	ATTENTION	✗	whether control was proven
G4	2.0	DECISIVE	✓	repos exist, are not forks, belong to or carry the account
G5	1.0	ATTENTION	✓	activity during the claimed project window
G6	1.1	DECISIVE	✓	claimed stack vs. the repository it links
G7	2.0	DECISIVE	✓	the repo contains a file a person wrote
G8	1.0	ATTENTION	✓	fork ratio across the account
G9	1.0	ATTENTION	✓	claimed tenure vs. earliest public evidence
G10	1.0	ATTENTION	✓	activity across a claimed employment window
G11	1.0	ATTENTION	✓	employers vs. public org membership

ID	VER	TIER	SCORES	WHAT IT CHECKS
P1	2.0	DECISIVE	✓	Codeforces rating
P2	4.0	DECISIVE	✓	contest rank, matched by contest **name**
P3	2.0	DECISIVE	✓	problems solved — Codeforces only
P4	1.0	DECISIVE	✓	LeetCode statistics
P5	1.0	ATTENTION	✓	CodeChef statistics vs. the public profile page (scraped)
P6	1.0	ATTENTION	✗	one handle across platforms
W1	3.0	DECISIVE	✓	deployments respond, **and whose they are**

Tier decides the bucket, mechanically

```
for f in findings:
    if f.tier is Tier.DECISIVE:    contradicted.append(f.finding_id)
    elif f.tier is Tier.ATTENTION: needs_context.append(f.finding_id)
```

No per-rule branching in the aggregator, and there must never be — that is the only thing stopping a judgement being presented as a fact.

A check that is technically correct can still be unfair

Sixteen rule versions would have put a false or misleading finding in front of a reviewer looking at an honest CV. None was a bug in the usual sense.

RULE	THE TEMPTING VERSION	WHY IT IS WRONG
G5	repo created after the claimed window	`created_at` bounds when the REPO existed, not when the WORK happened
G7	fewer than N files means empty	an 11 KB program in two files is a program
P2	is the rank anywhere in the history	true, useless, and it hid that the named contest was three places out
D9	department absent from the degree text	the CV named no department at all. Absence is not contradiction
G6	a claimed technology is missing	MongoDB is not a language and is not an npm package
G9	claimed tenure exceeds earliest evidence	the account is usually younger than the claim
G10	no public commits during a claimed job	that is what a salaried job looks like from outside
G11	employer absent from public orgs	org membership is private by default
D6	stated total exceeds the dated roles	a CV need not list every role; one page forces the choice
D5	any DEFINITE overlap between full-time roles	a **13-day** overlap is a notice period
G4	investigate the first owner the repo URLs name	a CV linking its own repos and `apache/kafka` had Apache checked as the candidate. The account is the stated handle, else the owner of most linked repos
D9	decode any token shaped like a roll number	every institution got IIT Kharagpur's department table. A scheme applies only where the CV names its institution as a whole word — "vit" is in "activities"
D10	two non-overlapping mentions of one role disagree	that is a return internship — two stints. It fires when ONE stint has two irreconcilable accounts
W1	a URL that does not answer 200 is dead	a timeout, a login wall or bot protection is not a 404. Only 404 and 410 are decisive

RULE	THE TEMPTING VERSION	WHY IT IS WRONG
P2	the claimed number appears in the span	as a substring, `10` is in "1106". Numbers are matched whole, here and in the grounding guard
G6	no stack match is a contradiction	with no manifest read, a language census cannot see a library. G6 reads all five manifest formats and abstains when it read none

Four survive as corroboration-only rules — G10, G11, P6 and the confirming half of G9. That is not a weakened check; it is the correct shape when absence is the ordinary state of an honest candidate.

Two worked examples

D5 — the handover band. A professional CV gave exact dates for the first time:

```
Curebase / Quotient    56 days, 22%  →  ✗ contradicted   (two months of dual full-time)
Quotient / Stewards    13 days,  5%  →  ? needs context  (a notice period)
```

Absolute days, not the fraction — a two-week handover between two one-month contracts is 50% of the shorter role and still a handover.

P2 — contest identity, platform-agnostic. The identity of a contest is its *largest* number; divisions and stages are small, and a year appears in the platform's record rather than in what a CV copies.

```
"Codeforces Round 998"          vs "Codeforces Round 998 (Div. 3)"   same
"Codeforces Round 998 (Div. 3)" vs "Codeforces Round 998"            same
"Meta Hacker Cup Round 2"       vs "Meta Hacker Cup 2024 Round 2"    same
"Codeforces Round 99"           vs "Codeforces Round 998 (Div. 3)"   different
"AtCoder Beginner Contest 350"  vs "AtCoder Beginner Contest 351"    different
```

11 The report

Five buckets, and the fifth is what makes the other four honest.

✓	**verified**	checked against an outside source, no discrepancy — plus corroboration notes
✗	**contradicted**	decisive findings, capped at 5, ranked by confidence
?	**needs context**	attention findings, **always printed with the innocent reading**
–	**not evaluated**	every abstention, with its reason
□	**out of scope**	the static exclusion list, always present

A reviewer gives a report about sixty seconds; the caps are a design constraint, not a preference.

`section_confidence` is a probability of contradiction, not a grade

```
combined = 1.0
for finding in group:
    weight = 1.0 if finding.tier is Tier.DECISIVE else ATTENTION_WEIGHT  # 0.4
    combined *= 1.0 - finding.confidence * weight
confidence = 1.0 - combined
```

Three things stop it becoming the score this system refuses to produce: non-scoring findings never contribute; a section with no scoring finding is **absent**, not zero — zero reads as "measured, found nothing" and gets averaged; and nothing is combined across sections.

> **The tier weight is not decoration.** `Discrepancy.confidence` says how sure we are of the *observation*: P2 recording rank 1109 against a claimed 1106 carries confidence 1.0, because the API definitely says 1109. Raw confidences drove an honest CV's achievements section to **1.0 on a three-place standings artefact**. Weighted, it reads 0.4.

Coverage describes the verification, not the candidate. A high number means "we checked thoroughly", never "they are good" — so it cannot become a hiring threshold.

12 The narrative

The second and last place a model is called. It does not write a briefing — it writes one sentence per finding, and Python assembles the frame.

OBSERVED

Asked for free prose over a report carrying four attention findings, the model opened with *"the CV does not contradict any rules, as all claimed information is verified"* — a verdict, from a system whose first principle is that it produces none. It passed a numeral check **because it contained no numbers.**

RESOLVED

A model that never writes a global sentence cannot write a false one. It now produces one sentence per finding; the counts, the ordering and the innocent readings are printed by Python.

LAYER	WHAT IT STOPS
signature	`narrate(report, provider)` — cannot reach the document, so cannot introduce a fact nobody computed
omission check	every finding must get a sentence — omission is how version 1 lied
containment	numerals checked **per finding**, not per report

Per-finding containment matters: a number belonging to P1 inside a sentence about G5 is the extraction failure arriving at the other end of the pipeline, and a report-wide check would pass it. Asked to include the benign reading, the real model copied the headline and dropped it — so that guarantee cannot depend on a model remembering.

13 The manual-check worksheet

Naming what a machine cannot do and stopping there is useless: a reviewer told "the degree was not verified" still has to work out which institution, which roll number, and what to ask.

`cvv verify --worksheet` emits the question, whom to ask, the roll number **already decoded**, and what the machine already established — and it invents nothing.

OBSERVED — ITS FIRST OUTPUT

It asked a candidate how two of nineteen proposed claims came to be dropped **by our extractor**, why their own placement cell wrote a disclaimer, and to account for a gap in their dates.

RESOLVED

A non-scoring finding never becomes a question for the candidate. `scoring=False` marks findings describing the extraction, the document, a career's shape or a naming habit — none is a claim the candidate made.

A verification contact is printed only where the operator supplied one in `data/institutions.json`, **which ships empty**: a guessed registrar address sends a candidate's details to a stranger, so an unlisted institution gets its office named by role instead.

14 The handoff to M6

M6, the skill graph, lives beside this module in `../skill-graph` and reads one file: the contract's `VerificationReport`, which `cvv verify --save DIR` writes as `verification-report.json` and `GET /v1/verifications/{id}` returns.

One function, `pipeline.verification_report()`, builds it for both, so the file and the response cannot drift apart. It computes nothing: status, coverage and provenance are read off the stages that produced them.

> **This module does not decide what evidence is worth.** An earlier `events.py` projected the report into M6's evidence events itself. M6 has its own projector, and what a verified artefact means for a skill is a skill-graph judgement — so ours was a second answer to M6's question, and it sent skill ids in a `surface:` namespace that M6 quarantined, all thirteen. It is gone.

The first real handoff failed at every step, and each failure was invisible until a real report crossed the boundary:

WHAT BROKE	WHY NOTHING CAUGHT IT
the report had no evidence bundle, subject, provenance or coverage	the contract test parsed the stages, never the whole report
M6 refused `document.created_at = D:20250704194743+05'30'`	jsonschema checks `format: date-time` only with an optional package
two fields were extensions nobody had listed	`Narrative` was not among the models the test compared

The contract test now builds a full report over real evidence and requires it to validate except for the named extensions — including inside `oneOf` branches, where a `github` block carrying extensions surfaces as "valid under none of the schemas".

Measured on the reference CV, live, through both modules:

```
cvv verify --save      59 s     4 artefacts, verification-report.json among them
M6 projection          35 events · 2 suppressions, each with a schema reason
                       platform_rating 1 (0.90) · claim_corroborated 6 · repo_artifact
                       artifact_live 3 · cv_claim 21
M6 ingest              35 accepted · 12 quarantined (tools and npm package lists)
M6 skill graph         23 skills as intervals; algorithms [0.564, 0.725]
```

`subject_ref` is never derived from the CV — minting an id from a name or roll number makes someone linkable across verifications they never connected. The default is `subject:<document_id>`, which links only re-runs of the same file.

15 Interfaces

```
cvv verify cv.pdf [--brief] [--worksheet] [--json] [--offline] [--save D] [--subject RE
cvv evaluate cv.pdf [--baseline D] [--save-baseline D]
cvv serve [--host] [--port]
cvv rules | inspect cv.pdf | config
```

HTTP — the integration spec, in one process

```
POST /v1/verifications              202 · Idempotency-Key honoured
GET  /v1/verifications/{id}         200 report · 202 while running · 404 unknown
GET  /v1/verifications/{id}/claims/{claim_id}/provenance
POST /v1/webhooks · GET /v1/usage · GET /healthz /readyz
POST /v1/narrate /evaluate · GET /v1/rules
POST /v1/resolve /attest            501, with the reason
```

The spec draws three services; this system collapsed them by explicit decision. **The two boundaries the split existed to enforce survive:** `/v1/narrate` accepts only a `FindingsReport`, `/v1/evaluate` only a `ClaimGraph`. Neither can reach the document, whether the call crosses a network or a stack frame.

DECISION	REASONING
No queue	`BackgroundTasks` runs the job here. Honest for one process, first thing to replace when it is not
`completed_degraded` must mean something	"any gap" was useless — before its scraper, CodeChef was a gap on *every* run, so every report was degraded. It now means something **transient** stopped a check
A failure is still a report	empty claims and findings, not absent — a client never needs two shapes, and a vanished verification leaves them with nothing to tell the candidate
501, not an empty success	an empty result reads as "resolved to nothing" rather than "never attempted"

Persistence — two stores, one Protocol

JSONB, not an ORM. The stored object is the report itself — a deep pydantic tree that already round-trips through JSON by contract — so shredding it into columns would create a second schema to keep in step with the first. Columns hold only what is *queried*, including the idempotency key under a **partial UNIQUE index**, because two concurrent retries of one upload cannot be settled by a check-then-insert.

OBSERVED — ONE SUITE, BOTH STORES

A duplicate idempotency key made memory quietly rebind the key to a **second verification**, while Postgres raised a `UniqueViolation` that reached the client as a **500**. Registering one webhook twice made memory fire it **twice** per verification and Postgres fire it once.

WHY IT WAS INVISIBLE

The Protocol makes mypy check they have the same *shape*. Only a suite running against both checks they have the same *behaviour* — and they did not.

Local, not hosted — decided. A stored report holds a real person's name, roll number, institutions and handles, so where the database lives is a question about consent rather than infrastructure. A hosted Postgres would need *no code change*, which is exactly why the decision has to be deliberate.

16 Cross-cutting

Logging never carries the CV

A log line is the easiest place for personal data to escape — written to disk, shipped to an aggregator, tailed in a shared terminal, kept long after the verification was deleted. Events carry **identifiers and counts**, never content.

```
REDACT = frozenset({
    "verbatim", "snippet", "text", "normalised_text", "raw", "document_text",
    "handle", "name", "email", "org", "title", "degree", "roll", "explanation",
    "summary", "prose", "claim_text", "url", "document_url",
})
```

A convention is not an enforcement mechanism, so `scrub` replaces those keys whatever a caller passed. Output goes to **stderr**, so `cvv verify --json > report.json` stays a file that parses.

Stable identifiers

```
claim_id   = "c" + sha1(f"{graph_id}|{char_start}|{kind}|{discriminator}")[:10]
finding_id = "f" + sha1(f"{report_id}|{rule_id}|{','.join(sorted(claim_refs))}")[:10]
```

The discriminator is load-bearing: one skills line yields several claims sharing a span and a kind, and without it they collide onto one id and all but one vanish.

17 How the system is measured

A unit test proves a rule does what its author intended, using a fixture that author wrote to exercise it. It cannot prove the system catches a falsified CV.

The mutation harness takes a real document that ran end to end, corrupts exactly one fact, and asks whether the check responsible notices. Seventeen mutations across 15 rules — two each for P2 and P5.

MEASURE	WHAT IT SAYS
Detection	the quotable number, and the easy one to inflate. "Something fired" does not count — the rule *responsible* must be the one that noticed
Collateral	rules firing on a mutation they had no business noticing. The honest one

Three refinements make the numbers honest: baseline findings are **subtracted** and named; a mutation the document cannot carry is **skipped, not failed**, or the rate measures the document rather than the system; and **an abstention is not a miss** — counting it as one would make detection a measure of how well-connected a candidate is.

What it found

- **P3 compared every `problems_solved` claim against Codeforces**, whatever platform it named. "Solved 800 problems on LeetCode" on an honest CV was decisively contradicted by a Codeforces count of 412 — the exact failure P3's own docstring warned about, committed one level up.
- **D11 reported a career gap between two mentions of the same role.** That is D10's finding; a document contradicting itself does not also have a gap.

Reproducibility, and its limit

Within one model load it is exact: four consecutive extractions of the same CV were byte-identical. **Across loads it is not.** Ollama picks the GPU/CPU layer split from whatever VRAM is free at load time — the same machine reported 100% GPU and later 81% in one session.

The filters absorb that variance *usually, not always*: an audit found a live extraction assertion failing once in three runs.

18 Deliberately not built

EXCLUDED	WHY
Entrance rank vs. cutoff	JEE cutoffs vary by reservation category; a category-blind comparison systematically flags reserved-category candidates for a rank that is entirely valid
AI-generated-CV detection	using a model to write a CV is not fraud, and detection penalises non-native English speakers hardest
Contribution authenticity (G12)	see below
Identity, employment, degree, CGPA	no accessible source. A degree can only be confirmed by the institution that awarded it
Any overall trust score	a number invites a threshold

G12 in detail — excluded, not backlogged

Cost was never the objection: GraphQL returns 100 commits a query, roughly 40 for a whole account.

It does not work in either direction. *Farming*: 400 commits with a median of one line is a cron job, and it is also `fix: typo in README` / `refactor: rename userId` / `test: add empty-input case` — the atomic-commit habit the industry teaches. *Bulk import*: one honest `npm install` writes `package-lock.json +14,203 −0`, which **is** the signature.

> **And it is shaped wrongly, which decides it.** Every other rule tests a *claim* — P1 tests "peak rating 1495", G7 tests "this is a project". G12 tests no claim: the CV never says its author's commits are of representative size. There is nothing to contradict, only a habit to be suspicious of. It is also defeated by `random.randint(1, 400)`, so it selects for naive GitHub usage — students, deadline pushes, and people whose real work is private.

19 Open decisions and known gaps

Nine fields ahead of the contract

Twenty-five of the twenty-seven schema definitions set `additionalProperties: false`, so a field added here makes our output **invalid** for a consumer validating against the schema. Nine exist, each load-bearing:

FIELD	NEEDED BY
`ExtractionMeta.filtered`	D2 2.0 — filtered ≠ ungroundable
`RepoEvidence.authored_files / authored_bytes`	G7 2.0 — a lock file is not substance
`RepoEvidence.homepage`	W1 2.0 — ties a deployment to its repo
`GitHubEvidence.public_repos / public_repo_count`	skills a CV cannot fit
`RuleOutcome.note`	what a corroborating rule confirmed
`Narrative.rejected_because`	why the model's prose was not used — never listed, because `Narrative` was not compared
`VerificationReport.failure`	why a verification failed; the contract has no field for it

A test holds them as a **named list** and fails on anything else — and also fails if an entry stops being an extension, so the list cannot rot. The integration spec predicted this failure mode verbatim: *"one person adds a field locally, the other never sees it, and integration breaks in week 7."*

Arguably the strongest reading is that the spec itself prescribes `schemas/*.schema.json — generated from Pydantic, committed`, which makes the models the source of truth and the upstream draft what should be regenerated.

Honest gaps

- **No gold set.** Phase 5 specifies two instruments: a mutation engine (built) and a gold set measuring extraction precision and recall per claim kind (not built). **Extraction accuracy is unmeasured**; only rule detection is.

- **Three documents is a small corpus**, and one is synthetic. Each new document has found a real bug within minutes.
- **A small model.** Several documented failures are `qwen3:1.7b`'s limits rather than the design's, and a larger model is the intended upgrade path.
- **G6 tier 2** is deferred by decision: reading repository files opens a prompt-injection surface whose contents the candidate chooses, for a gain the manifest check already approximates.

20 The invariant

> A CV from someone with no account, no repositories and no contest history must still produce a complete, honest report.

All eleven document-intrinsic checks need no network at all, so the document-only path is a real path rather than a failure mode. `cvv verify --offline` is that path, and a test asserts every rule still appears in the report with its reason.

Everything else follows from this: **abstention is a first-class result**, and evidence gathering lives inside this system rather than arriving from somewhere else.

21 A worked example, end to end

One document, every stage, with the real intermediate state. Nothing below is reconstructed — each block is the actual output of the stage above it.

> **Run on 15 September, before P5 and the October fairness fixes.** A run today differs in three places: the CodeChef gap in stage 06 is P5 corroborating a highest rating of 1746 against the claimed 1733; P6 corroborates one handle, `ar009`, on Codeforces and CodeChef; and coverage is 16 of 28 evaluated, 57%.

STAGE 01

Ingest

```
doc_0906632773e2   1 page   5,644 chars   ingest=pdf_text
producer    iText 2.1.7 by 1T3XT
created     D:20250704194743+05'30'

sections detected:
  EDUCATION                     79..259      180 chars
  PROJECTS                     259..3361    3102 chars
  SKILLS AND EXPERTISE        3361..3834     473 chars
  COURSEWORK INFORMATION      3834..4331     497 chars
  AWARDS AND ACHIEVEMENTS     4331..5148     817 chars
  EXTRA CURRICULAR ACTIVITIES 5148..5644     496 chars
```

The education table has already collapsed into one line — `2027 B.TECH IIT Kharagpur 9.34 / 10` — which is what makes `line_at` provenance work later. `producer = iText` is retained for D4 and is **trust-increasing**: a CV generated by a placement cell's template went through that cell.

STAGE 02

The regex pre-pass

```
identifier   '23CS10004'
profile      'github.com/Ashutosh-Ranjan09/ATMA-Final'
```

Two things, both with a grammar rather than a convention. Every other pattern this module once carried is now a model pass.

STAGE 03

Routing

```
achievements   4331..5148    817 chars   'AWARDS AND ACHIEVEMENTS • Secured …'
education        79..259     180 chars   'EDUCATION Year Degree/Exam Institu…'
projects        259..3361   3102 chars   'PROJECTS Automatic Timetable Manag…'
skills         3361..3834    473 chars   'SKILLS AND EXPERTISE • Programming…'
```

`COURSEWORK INFORMATION` routed to `coursework` and `EXTRA CURRICULAR ACTIVITIES` to `other` — **neither is consumed by any pass.** That is the fix from §6: with no `coursework` category, the education pass turned "Computer Science" into a degree awarded by "University", and D9 picked that one.

STAGE 04

What the passes proposed

Six proposals from the 817-character achievements window:

```
platform='codeforces' metric='contest_rank' value=873  contest='JEE Advanced 2023' hand
platform='codeforces' metric='contest_rank' value=809  contest='JEE Main 2023'     hand
platform='codeforces' metric='max_rating'   value=1495 qualifier='Specialist'      hand
platform='codechef'   metric='max_rating'   value=1733 qualifier='3-Star'          hand
platform='codeforces' metric='contest_rank' value=1106 contest='Codeforces Round 998'
platform='leetcode'   metric='problems_solved' value=500
```

> **Two guardrail violations are visible in that block**, and neither is fixable by prompting.
>
> Rows 1 and 2 are **JEE ranks labelled** `platform='codeforces'`. The design excludes entrance ranks on fairness grounds and the prompt says so; the model returned them anyway, relabelled as a platform it does understand. No label-based filter could catch this.
>
> The model also stamped `handle='AR009'` onto both JEE rows, carried over from the Codeforces sentence — the "one answer pasted onto several spans" failure in miniature.

The **handles** pass returned both accounts with the document's own casing — `AR009` and `ar009`. **declarations** found the CDC footer in the bottom fifth of the page. **summary** returned nothing, which is the honest answer for a student CV and the answer the pass is built to be able to give.

STAGE 05

Grounding and the gate

```
proposed 22  →  kept 20      span_failures=0      filtered={'excluded_exam': 2}
```

Both JEE claims are gone — **dropped by reading the verbatim**, not the label the model supplied. The surviving graph:

CLAIM	KIND	KEY FIELDS
`c2abb329f17`	identifier	`23CS10004 → admission 2023, dept CS, serial 10004`
`c59d74e1b45`	education	B.Tech · IIT Kharagpur · 2027 (year precision, a point)
`c2990167c81`	education	CBSE Class XII · Krishna Public School · 2023
`cb8a75a6ead`	education	CBSE Class X · Open Minds A Birla School · 2021
`c60d8614fec`	project	ATMA · IIT Kharagpur · 8 technologies · 2025-03 … 2025-04
`c7440d7fd61`	project	CF-Connect · 7 technologies · 2025-05 … 2025-06
`c14539ba26d`	project	Dijkstra Visualizer · 9 technologies · 2025-06 (a point)
`ca4f6b6adca`	project	RSA Chat Application · 6 technologies · 2024-06
`c25c1bf1ef7`	profile	github · `Ashutosh-Ranjan09`
`c81fcf2943d`	profile	codeforces · `AR009`
`c4a8c311f17`	profile	codechef · **`ar009`**
`c4ac467314f`	achievement	codeforces · max_rating **1495** · Specialist
`cbd9f4d98c2`	achievement	codechef · max_rating **1733** · 3-Star
`cfc8cb653ac`	achievement	codeforces · contest_rank **1106** · `contest='Codeforces Round 998'`
`cc86bcc0b93`	achievement	leetcode · problems_solved 500 · `flags=['aggregate_across_sources']`
`c3f51447bdd` +2	skill	9, 7 and 6 technologies, one claim per source line
`c9f92f7ff95`	declaration_marker	the CDC footer

Four gate decisions are visible: the **CodeChef handle survived as `ar009`** though `AR009` sits two lines above it; the **500-problem claim is flagged, not dropped**, because its sentence names three platforms; the junk employment claim was discarded by the span-conflict stage; and the contest name reached `context` only because it appears in that claim's own span.

STAGE 06

Evidence gathered

```
codeforces   AR009              found   max_rating=1495  solved=164  contests=17
github       Ashutosh-Ranjan09  ownership=claimed_unverified  created=2024-06-29
                                public_repos=7   contribution_windows=28
  ATMA-Final           fork=False  created=2025-06-24   56 files / 150,238 B  [JavaScri
                                   homepage=https://atma-ar.vercel.app
  Dijkstra-visualizer  fork=False  created=2025-06-24   14 files /  79,944 B  [JavaScri
  Terminal-Chat-Room   fork=False  created=2024-06-30    3 files /  11,796 B  [Python]
  cf-connect           fork=False  created=2025-05-09  111 files / 452,428 B  [TypeScri
deploy       atma-ar.vercel.app          200
deploy       cf-connect.vercel.app       200
deploy       dijkstra-visual.vercel.app  200
GAP          codechef  not_supplied  "no API — a scraper is the only route"
```

Two of the highlighted figures are the ones that produced guardrail 9. **`Terminal-Chat-Room`** is the three-file, 11 KB program G7's first version would have called an empty repository. **`ATMA-Final` was created 2025-06-24**, two months *after* the project's claimed Mar–Apr window — the comparison G5's first version would have reported as a contradiction, on a course project pushed after the deadline.

STAGE 07

All 27 rules

FIRED	CLEAN	ABSTAINED
D2 D3 G3 P2	D4 D7 G2 G4 G5 G6 G7 G8 P1 W1	D5 D6 D8 D9 D10 D11 D12 G9 G10 G11 P3 P4 P6

Every abstention carries its reason. The most informative:

```
D9   the CV states no department to compare 23CS10004 against.
     Checked: 23CS10004 encodes admission in 2023 and a 4-year B.Tech completes
     in 2027, which the CV agrees with.
P3   The claim aggregates 3 platforms (codeforces, geeksforgeeks, leetcode);
     no single source can confirm or contradict it.
D5   Fewer than two dated roles are claimed; there is nothing to overlap.
G11  The CV names no employer to look for.
```

> **D9 is the shape the whole design argues for.** Half the check ran and half could not, and it says which. "Could not check the roll number" and "checked the years, they line up, could not check the department" are different statements, and only one of them is true.

STAGE 08

Findings

```
[P2] attention · info · conf 1.0 · scoring=True
     The CV claims rank 1106 in Codeforces Round 998 (Div. 3). The Codeforces API
     records rank 1109 for 'AR009' in that contest — 3 places lower.
     though: Ranks shift after system testing and after cheat removals.

[G3] attention · low  · conf 0.55 · scoring=False   ownership never proven
[D2] attention · low  · conf 1.0  · scoring=False   2 of 22 refused by the filters
[D3] attention · info · conf 1.0  · scoring=False   the CDC disclaimer
```

P2 is the one real signal, and it is the whole design in one line: the API was matched **by contest name**, both numbers are stated, the gap is three places on 1106, and the benign reading travels with it. Its earlier version reported "no contest records rank 1106" — true, useless, and it hid that the named contest was found.

STAGE 09

Section confidence


```
[{'section': 'achievements', 'confidence': 0.4, 'basis_finding_ids': ['fbb5923211f']}]
```


One entry. G3, D2 and D3 are `scoring=False` and contribute nothing; P2 contributes `1.0 × 0.4` because it is attention-tier. **No other section appears** — absence means no scoring finding touched it, which is not the same as zero.

STAGE 10

The buckets

✓	**verified · 7**	rating · deployment-to-repo linkage · account · repos · substance · stack · no anachronism
✗	**contradicted · 0**	nothing decisive
?	**needs context · 4**	P2 · G3 · D2 · D3, each with its innocent reading
–	**not evaluated · 14**	every abstention, with its reason
□	**out of scope · 4**	degree · employment · identity · entrance rank

One of those seven verified lines is a `Corroborated` note rather than a bare pass: W1 found the repositories' own declared homepages matching the claimed URLs — the difference between "this URL responds" and "this URL is theirs".

Coverage: 14 of 27 evaluated — 52%, printed on the first screen, alongside what the number means: it describes the verification, not the candidate.

STAGE 11

The briefing

```
0 contradicted · 4 needing context · 7 corroborated · 14 not evaluated.
No score is produced.

  [P2] The CV claims rank 1106 in Codeforces Round 998 (Div. 3). The Codeforces
       API records rank 1109 for 'AR009' in that contest — 3 places lower.
       though: Ranks shift after system testing and after cheat removals, so a
       CV written from the live standings can be a few places out.
```

Every sentence passed per-finding numeral containment. The counts, the ordering and every `though:` line are Python's.

OBSERVED WHILE WRITING THIS SECTION

The model prefixed every sentence with the rule id — `"P2 says the CV claims…"` — which renders as `[P2] P2 says …`. Adding "do not name the rule inside the sentence" to the prompt changed nothing, on two separate models.

GUARDRAIL 11, AGAIN

Stripped by `strip_rule_prefix`, in code the model cannot reach. Only a *leading* mention goes — a rule id inside a sentence is a citation and stays.

STAGE 12

The M6 projection

This used `events.py`, removed on 25 October; M6's own projector does this now — see §14.

```
11 events, 5 withheld

platform_rating  positive  base 0.90 → 0.90   []
repo_artifact    positive  base 0.75 → 0.375  ['ownership_unverified']   × 4
artifact_live    positive  base 0.60 → 0.60   []                          × 3
cv_claim         positive  base 0.20 → 0.12   ['undated_observation']     × 3

withheld:  non_scoring_rule × 3 · not_a_capability_signal × 1 · institution_disclaimed
```

The asymmetry as two numbers. Codeforces is **uncapped at 0.90** — the handle resolved on the platform's own API, so ownership is not in question. Every GitHub repository is **halved to 0.375**, because control of that account was never proven and this is corroborating evidence. A *contradicting* event from the same account would not have been capped.

`institution_disclaimed` is the CDC footer scoping the whole document: forwarding those claims as capability signals would launder a caveat the issuer wrote.

STAGE 13 **The worksheet**

```
ASK THE INSTITUTION
  [MC-c59d74e1b45]  of: IIT Kharagpur — academic / examinations section
      Confirm that B.TECH was awarded (the CV states 2027), and confirm the CGPA.
      · as written on the CV: '2027 B.TECH IIT Kharagpur 9.34 / 10'
      · roll 23CS10004 decodes to admission year 2023, department CS, serial 10004

ASK THE CANDIDATE
  [MC-fbb5923211f]  The CV claims rank 1106 … records 1109. Ask how this came about.
  [MC-ownership]    Prove control of github.com/Ashutosh-Ranjan09 — a gist, a commit,
                    or a profile-bio string YOU supply.

OPEN AND LOOK
  [MC-atma-ar.vercel.app]  Open it and confirm it does what ATMA describes.   (W2)

JUDGEMENT ONLY — no machine result exists or should
  [H1] Does the work match the claimed seniority?   [H2] Is the portfolio substantial?
  [H3] Code quality in context                      [H6] The hiring decision
```

The roll number arrives **already decoded**, so a reviewer phoning the institution does not have to work out what `23CS10004` means. And D2, D3 and G3 — all `scoring=False` — produce **no question for the candidate**: nobody should be asked why our extractor dropped a claim, or why their own placement cell wrote a disclaimer.

STAGE 14

What the run cost

PHASE	WORK	TIME
extraction	8 model calls + 1 routing call	≈ 22 s
gathering	~20 HTTP requests	≈ 8 s cold · <1 s warm
rules	27 checks	≈ 3 ms

The model is the entire budget. Everything after extraction is deterministic and effectively free — which is the reason the boundary sits exactly where it does.
